

Task sheet for module “Numerical root-finding”

Submission deadline: October 7 at 23:59

Instructions: Make a serious attempt at all problems below (with the exception of the bonus tasks, which are elective). Submit your work (PY code + PDF answers = ZIP archive) at <https://kurser.math.su.se/mod/assign/view.php?id=106504>. Then you'll be asked to present your work on Thursday, October 8 at 10:00. No slides needed!

Collaboration: Feel free to work on the problems together, but: all should (i) submit their work on the course page (even if repeated) and (ii) be able to discuss what they submitted.

Task 1 (Contour-integration methods). Objective: find all roots of

$$f(z) = e^{2z} \cos(5z) - 5z^2$$

inside the unit disk $K = \{z \in \mathbb{C} : |z| < 1\}$.

You will need to use quadrature rules to approximate contour integrals in the complex plane. I suggest you implement the simple and accurate (periodic-)trapezoidal rule

$$\frac{1}{2\pi i} \oint_{\partial K} g(z) dz = \left[\begin{array}{l} z = e^{it} \\ dz = ie^{it} dt \end{array} \right] = \frac{1}{2\pi i} \int_0^{2\pi} g(e^{it}) ie^{it} dt \approx \sum_{j=1}^M g(e^{it_j}) \frac{e^{it_j}}{M}, \text{ with } t_j = \frac{2\pi j}{M}.$$

- (a) How many roots does f have in K ? Answer (through Cauchy's argument principle) by using a quadrature formula to approximate the contour integral

$$N = \# \text{ of roots of } f = \frac{1}{2\pi i} \oint_{\partial K} \frac{f'(z)}{f(z)} dz \approx \sum_{j=1}^M \frac{f'(e^{it_j})}{f(e^{it_j})} \frac{e^{it_j}}{M}.$$

Hint: you need a large enough number of quadrature points M .

- (b) Implement and run the Delves-Lyness / Beyn-Hankel root-finder:

- Through quadrature, compute moments $S_p = \frac{1}{2\pi i} \oint_{\partial K} z^p \frac{f'(z)}{f(z)} dz$ for $p = 0, \dots, 2N-1$.
- Construct the two Hankel matrices $H_0, H_1 \in \mathbb{C}^{N \times N}$: $H_0 = (S_{i+j})_{i,j=0}^{N-1}$ and $H_1 = (S_{i+j+1})_{i,j=0}^{N-1}$.
- Compute the roots as generalized eigenvalues $H_1 v = \lambda H_0 v$ using `scipy.linalg.eig`.

Once you have the approximate roots $\{r_j\}_j$, plot them and compute their *residuals* $\{|f(r_j)|\}_j$. Are they small?

- (c) A simple way to improve accuracy is to follow up the DL/BH method with Newton's method! For each approximate root r_j , run Newton's method (`scipy.optimize.newton`) starting with guess $z_0 := r_j$. Does the approximation quality get better?
- (d) If the number N of roots is not available, one must guess it ($\tilde{N}$) and compute $2\tilde{N}$ moments in the DL/BH method, which results in different Hankel-matrix sizes. Try to apply DL/BH with fewer and more moments than the correct amount ($2N$). For each guess $\tilde{N}$, plot the approximate roots and the condition number $\kappa(H_0)$ (`numpy.linalg.cond`).
- (e) **Bonus:** solve the problem replacing the DL/BH method with Newton's method with deflation. Report your findings.

Task 2 (Black-box solution of a nonlinear eigenvalue problem (NLEVP)). Objective: solve an NLEVP on the rectangle $K = \{z \in \mathbb{C} : 12500 < \operatorname{Re} z < 112500, 0 < \operatorname{Im} z < 10000\}$.

The NLEVP (https://guettel.com/rktoolbox/examples/html/example_nlep.html#4) is of the form

$$\text{find all } \lambda \in K \quad : \quad f(z) := \det \underbrace{\left(S - \lambda M + i\sqrt{\lambda - \sigma_1} W_1 + i\sqrt{\lambda - \sigma_2} W_2 \right)}_{F(z)} = 0, \quad (1)$$

where S , M , W_1 , and W_2 are large (but sparse) square matrices, σ_1 and σ_2 are given real scalars, and $\sqrt{\cdot}$ are some fixed branches of the complex square root. **Note:** σ_1 and σ_2 are outside K so that f and F are analytic on K .

Instead of solving the above problem (the determinant is so expensive!), we consider

$$\text{find all } \lambda \in K \quad : \quad \lim_{z \rightarrow \lambda} \underbrace{\left(v^\top F(z)^{-1} w \right)}_{g(z)} = \infty, \quad (2)$$

with v and w some fixed vectors with as many entries as K has rows. Since g (scalar-valued) has much smaller size than $F(\cdot)^{-1}$ (matrix-valued), this approach is called “sketching”.

You are provided with a script `g_nlep.py` implementing class `g_nlep`, which can evaluate $g(z)$ at any point $z \in \mathbb{C}$: after initializing `g=g_nlep()`, which loads matrices S , M , W_1 , and W_2 , as well as vectors v and w , evaluation may be carried out through a direct call `g(z)`.

- (a) Prove that any λ solving (2) must also solve (1). Argue why the reverse may not always be true (hint: it may depend on v and w).
- (b) We now want to set up a finite set $\{z_j\}_{j=1}^J$ of sampling points within $\overline{K}$ and evaluate g at each of them, which yields the dataset $\{(z_j, g(z_j))\}_{j=1}^J \subset \mathbb{C}^2$.

If you try calling `g_nlep`, you will realize that this takes a long time, especially if J is not small! So for convenience you are provided with the CSV file `g_eval.csv`, which contains some data I precomputed for you: the first two columns contain real and imaginary parts of z_j , while the last two columns contain real and imaginary parts of $g(z_j)$.

Load the data from `g_eval.csv` (`numpy.loadtxt` or similar) and build two complex vectors `z_eval` and `g_eval`, each of length J . Apply the AAA algorithm (`scipy.interpolate.AAA`) to $g(z)$ to construct an approximation $\mathbf{r}(z) \approx g(z)$.

- (c) Extract the poles $\{\lambda_j\}_j$ of $\mathbf{r}$ through `r.poles()` and plot them. For physical reasons, no eigenvalues can have negative imaginary part. Do you get any of those?
You can also certify the poles’ correctness by computing the residuals $\{1/|g(\lambda_j)|\}_j$. Which poles have small residuals? Can the poles outside K be trusted?
- (d) Qualitatively (no code needed!) compare the AAA-based approach to the DL/BH method from Task 1. What are pros and cons of each, especially when applied to a problem where evaluating f (or g here) is very expensive? What determines the complexity of each method?

- (e) **Bonus:** one of the nice things about surrogate-based root-finding is that one can always make the surrogate better, although the computational price might be high. A simple way to improve a rational surrogate is to add more samples to the dataset used by AAA, i.e., increase J , especially if the sample points are placed close to the poles of the target function g ! Add some more (expensive) sample points near all $\{\lambda_j\}_j$ within K and run AAA on the extended dataset. Report your findings.